



Timetable Scheduler

Sessional II Assignment
for
Artificial Intelligence (CO401)

Prepared by

Debasish Bordoloi (CSB22003)
Sakil Aziz (CSB22012)
Debasish Bhuyan (CSB22023)
Bhaskar Jyoti Hazarika (CSB22031)
Yashwant Kumar Upadhyay (CSB22064)
Mriganku Krishan Yogi (CSB22079)

*7th Semester, Bachelor of Technology,
Department of Computer Science & Engineering,
Tezpur University*

Submitted to

Dr. Shobhanjana Kalita
*Assistant Professor,
Department of Computer Science & Engineering,
Tezpur University*

November, 2025

Abstract

The scheduling of academic timetables is a classic formulation of a Constraint Satisfaction Problem (CSP) and Constraint Optimization Problem (COP). It involves assigning resources (instructors, rooms, etc.) to tasks (classes) at specific times while adhering to strict rules. This project proposes an automated Timetable Scheduler that utilizes a two-phase Artificial Intelligence approach. Phase I focuses on generating valid solutions that satisfy all *hard* constraints (e.g., preventing double-booking of professors). Phase II employs an evolutionary local search algorithm to refine these valid schedules against *soft* constraints (e.g., minimizing gaps between classes, avoiding early morning saturation). The result is a system capable of producing a conflict-free, human-friendly academic schedule.

1 Introduction

1.1 Problem Statement

Manual timetable generation is a tedious, error-prone task that grows exponentially in complexity with the number of subjects, batches, and faculty members. A simple "legal" schedule often fails to consider the comfort of the students and faculty, leading to issues like erratic gaps between lectures or exhausting consecutive sessions.

1.2 Objectives

The primary objective is to develop a software agent that generates an optimized timetable. The system aims to optimize the following *soft constraints*:

1. **Gap Minimization:** Reducing idle time between classes for students.
2. **Load Balancing:** Avoiding cases where only a single class exists in a forenoon or afternoon session (minimizing travel for just one hour).
3. **Time Preference:** Minimizing the frequency of early morning classes, depending on configuration.
4. **Subject Distribution:** Preventing the concentration of the same subject on a single day (e.g., avoiding three Math lectures on Monday).

2 System Methodology

Our solution architecture is divided into two distinct processing phases.

2.1 Phase I: Constraint Satisfaction (Validity)

The first phase addresses the Hard Constraints using a backtracking search. A hard constraint is a condition that must be met for the timetable to be considered valid.

Instructor Conflict A professor cannot teach two different batches at the same time.

Batch Conflict A student batch cannot have two classes assigned to the same time slot.

Algorithm: Randomized Backtracking The system reads batch and course data from CSV files. It attempts to place classes recursively:

1. Select a class (Subject-Batch pair) from the list of classes to schedule.
2. Shuffle available time slots to ensure randomness.
3. Attempt to place the class in a slot.
4. Check Hard Constraints (is the slot empty? is the instructor free?).
5. If valid, recursively attempt to schedule the next class.

6. If a dead-end is reached, backtrack (remove the class) and try the next slot.
7. This process repeats until K valid timetables are found or the search space is exhausted.

2.2 Phase II: Evolutionary Optimization

The second phase treats the schedule as an optimization problem. It takes the valid timetables from Phase I and evolves them to minimize a penalty score.

Algorithm Steps

1. **Initialization:** Load the K valid timetables into a Min-Heap (priority queue) based on their penalty score.
2. **Mutation:**
 - Extract the best schedule from the heap.
 - Perform a Mutation: Select two random batches and two random slots. Swap the classes between these slots (or move a class to an empty slot) only if the swap does not violate hard constraints (check for instructor /batch clashes).
 - This generates a new candidate timetable.
 - Repeating this for K timetables, will result in NK timetables.
3. **Selection:** Any remaining timetables inside the heap are first deleted. These deleted ones are essentially the timetables rejected by the evolutionary process. Both the original and the mutated timetables are pushed back into the heap. The heap property ensures the best-performing schedules are always prioritized for the next iteration.
4. **Termination:** The system uses a **patience** counter.
 - If the best score in the heap improves, patience is reset to **max_patience**.
 - If the score does not improve, patience is decremented.
 - The process terminates when **patience** reaches 0.

3 PEAS Description

To formally define the Artificial Intelligence agent, we analyze the Performance, Environment, Actuators, and Sensors (PEAS).

3.1 Performance Measures

The agent is evaluated on a multiplicative penalty-based scoring system. Lower scores indicate better performance. The score is calculated as:

$$score = (1 + P_{gaps}) \times (1 + P_{single}) \times (1 + P_{morning}) * (1 + P_{concentration})$$

where:

- P_{gaps} : Cumulative count of empty slots between classes in a day.
- P_{single} : Penalty added if a batch has exactly one class in the pre-recess or post-recess session (heavier penalty for afternoon single classes).
- $P_{morning}$: Count of classes scheduled in the very first slot of the day.
- $P_{concentration}$: Penalty based on the frequency of the same subject appearing multiple times in one day.

3.2 Environment

3.2.1 Entities

The environment includes:

- List of subjects/courses to be scheduled.
- List of batches/student groups.
- List of professors/teachers.
- Absolute constraints (e.g., professor availability, specific class durations, classroom availability, etc.).
- The objective function and optimization goals.
- The set of K or NK candidate timetables that are being generated and evaluated during Phase I and Phase II.

3.2.2 Environment Categorization

- **Fully Observable:** The agent has access to the entire state of the timetable.
- **Deterministic:** The outcome of a specific swap is known; the randomness lies only in the selection of the swap targets.
- **Sequential:** Mutations are applied iteratively to previous states.
- **Static:** The class requirements do not change during execution.
- **Discrete:** Time slots are discrete, discontinuous.
- **Single-Agent:** Only one agent is working in the environment.

3.3 Actuators

The actuators in our problem setup are:

- **Timetable Generation:** Placing a specific Subject-Batch-Professor triplet into a specific Day-Time slot (Phase I).
- **Evolutionary Changes (Swaps):** Swapping the positions of two scheduled classes within the timetable (Phase II).

3.4 Sensors

The sensors in our setup are:

- **Absolute Constraint Conflict Detector:** Senses the existence of an absolute constraint violation (e.g., two classes for the same professor at the same time).
- **Objective Function Evaluator:** Senses the score/penalty of a given candidate timetable against the optimization goals (gaps, single-class sessions, early morning classes, subject concentration).
- **Timetable State Reader:** Senses the current arrangement (all scheduled classes and their slots) of the timetable being evaluated.
- **Termination Condition Reader:** Senses the number of iterations P since the last improvement to decide when to stop the Phase II process.

4 Implementation Details

The logic of the time table scheduler has been written using Python. We have also used the following Python libraries for different ends:

- `csv`, `io`, `os`: For file handling and data ingestion.
- `random`: For stochastic selection in backtracking and mutation.
- `heapq`: For efficient priority queue management in the genetic algorithm.
- `pandas`: For formatting the final output into a readable DataFrame.
- `streamlit`: For the user interface and progress visualization.

5 Conclusion

This project implements a robust academic scheduler. By combining Randomized Backtracking for validity with a Heap-based Genetic Algorithm for optimization, the system effectively balances strict requirements with user comfort. The multiplicative scoring model ensures that a schedule must be "good enough" across all soft constraints rather than just optimizing one at the expense of others.